

TẦNG 1. PRESENTATION LAYER

```
class AppSessionState <<global_state>> {
    + system_initialized: bool
    + ad_closed: bool
    + is_logged_in: bool
    + user_role: String
    + username: String
    + user_obj: UserData
    + payment_step: bool
    + current_page: String
    + selected_movie: String
    + selected_seats: List<String>
    + target_date: String
    + target_time: String
    + generated_ticket_ids: List<String>
    + payment_start_time: double
    + booking_success: bool
    + recent_tickets: List<String>
    + config_slider: List<String>
    + config_list: List<String>

    // Lưu trữ Controllers để giữ trạng thái
    + auth_ctrl: AuthController
    + movie_ctrl: MovieController
    + showtime_ctrl: ShowtimeController
    + room_ctrl: RoomController
    + booking_ctrl: BookingController
    + admin_ctrl: AdminController
}
```

```
class MainMenu <<boundary>> {
    + render_login_tab(): void
    + render_register_tab(): void
}
```

```
class AdminUI <<page>> {
    + view_dashboard_metrics(): void
    + manage_movie_tab(): void
    + manage_showtime_tab(): void
    + manage_room_tab(): void
    + config_display_tab(): void
    + view_top_revenue_tab(): void
}
```

```

+ sell_ticket_offline_tab(): void
+ manage_tickets_tab(): void
}

```

```

class CustomerUI <<page>> {
+ view_home_page(): void
+ view_booking_page(): void
+ view_history_page(): void
}

```

```

// Hàm phụ trợ (ui_components.py)
+ navigate_to(page: String, movie: String = ""): void
+ show_advertisement(): void
+ show_popcorn_effect(): void
+ create_premium_movie_card(col: Any, title: String, genre: String, duration: int, price:
double, img_url: String): void

```

TẦNG 2. BUSINESS LOGIC LAYER

```

class GlobalState <<factory>> {
+ init_global_system(): Tuple<AuthController, MovieController, ShowtimeController,
RoomController, BookingController, AdminController>
}

```

```

class AuthController {
- _io_handler: FileIOHandler
- _user_table: UserHashTable
- _current_user: UserData
- _auth_lock: Lock

- _generate_user_id(): String
+ login(username: String, pass: String): String
+ register(username: String, pass: String, confirm_password: String): bool
+ logout(): void
+ get_current_user(): UserData
+ is_logged_in(): bool
+ is_admin(): bool
+ get_user_table(): UserHashTable
+ get_all_users(): List<UserData>
}

```

```

class AdminController {

```

```

- _movie_controller: MovieController
- _booking_controller: BookingController
- _showtime_controller: ShowtimeController
- _room_controller: RoomController

+ calculate_revenue(): double
+ count_movies(): int
+ count_tickets(): int
+ get_top_movies_by_revenue(limit: int = 10): List<MovieData>
+ sell_ticket_at_counter(movie: MovieData, showtime: Showtime, row: int, col: int):
bool/String
+ generate_report(): Map<String, Any>
+ get_active_tickets(): List<TicketData>
+ admin_delete_movie(movie_id: String): bool
+ admin_delete_room(room_id: String): bool
}

class MovieController {
- _io_handler: FileIOHandler
- _movie_list: MovieLinkedList

+ generate_movie_id(): String
+ add_movie(movie: MovieData): bool
+ update_movie(movie_id: String, title: String, genre: String, duration: int, description:
String, base_price: float, poster_path: String): bool
+ delete_movie(movie_id: String): bool
+ search_by_id(movie_id: String): MovieNode
+ search_by_title(title: String): MovieNode
+ get_movie_list(): MovieLinkedList
+ get_movie_data(): List<MovieData>
+ movie_exists(movie_id: String): bool
+ save_data(): void
+ load_ui_config(): Map<String, List<String>>
+ save_ui_config(slider_titles: List<String>, list_titles: List<String>): void
}

class BookingController {
- _io_handler: FileIOHandler
- _showtime_controller: ShowtimeController
- _movie_controller: MovieController
- _room_controller: RoomController
- _ticket_list: TicketLinkedList
- _booking_lock: Lock

```

```

+ parse_seat_id(seat_id: String): Pair<int, int>
- _generate_ticket_id(): String
- _sync_matrix_from_tickets(): void
- _refresh_booking_data_no_lock(): void
+ process_booking(user: UserData, movie: MovieData, showtime: Showtime, seats:
List<Pair<int,int>>): List<String>
+ confirm_bookings_bulk(ticket_ids: List<String>): bool
+ process_counter_booking(movie: MovieData, showtime: Showtime, row: int, col: int):
bool/String
+ get_booking_history(user_id: String): List<TicketData>
+ find_ticket(ticket_id: String): TicketNode
+ generate_ticket_info(ticket_id: String): String
+ get_ticket_list(): TicketLinkedList
+ get_ticket_data(): List<TicketData>
+ cleanup_unfinished_reservations(timeout_minutes: int = 5): void
+ admin_cancel_ticket(ticket_id: String): bool
+ refresh_booking_data(): void
}

```

```

class ShowtimeController {
- _io_handler: FileIOHandler
- _movie_controller: MovieController
- _showtime_list: ShowtimeLinkedList

+ generate_showtime_id(): String
+ add_showtime(st: Showtime): bool
+ update_showtime(showtime_id: String, new_start_time: String): bool
+ delete_showtime(showtime_id: String, ticket_controller: Any = None): bool
+ find_showtime(showtime_id: String): ShowtimeNode
+ check_seat_status(showtime_id: String, row: int, col: int): bool
+ book_seat(showtime_id: String, row: int, col: int): bool
+ release_seat(showtime_id: String, row: int, col: int): bool
+ get_showtime_list(): ShowtimeLinkedList
+ get_showtime_data(): List<Showtime>
+ change_seat_status_by_admin(showtime_id: String, row: int, col: int, new_status: int):
bool
+ get_schedule_by_date(target_date: String): List<List<Any>>
+ extract_date(showtime: Showtime): String
+ extract_time(showtime: Showtime): String
+ get_unique_sorted_dates(showtimes_list: List<Showtime>): List<String>
+ get_unique_sorted_times(showtimes_list: List<Showtime>): List<String>
+ get_showtimes_by_movie(movie_id: String): List<Showtime>
+ find_exact_showtime(movie_id: String, target_date: String, target_time: String):
Showtime

```

```
}
```

```
class RoomController {  
    - _io_handler: FileIOHandler  
    - _room_list: RoomLinkedList  
  
    + add_room(room: Room): bool  
    + update_room(room_id: String, new_name: String): bool  
    + delete_room(room_id: String): bool  
    + find_room(room_id: String): RoomNode  
    + find_room_by_name(room_name: String): Room  
    + get_room_list(): RoomLinkedList  
    + get_room_data(): List<Room>  
    + count_rooms(): int  
}
```

TẦNG 3. DATA STRUCTURE LAYER

```
class Node {  
    + data: Any  
    + next: Node  
    + get_data(): Any  
    + set_data(data: Any): void  
    + get_next(): Node  
    + set_next(next_node: Node): void  
}
```

```
class UserNode <<node>> {}           // Các hàm thực thể kế thừa hàm Node cha  
class TicketNode <<node>> {}  
class MovieNode <<node>> {}  
class ShowtimeNode <<node>> {}  
class RoomNode <<node>> {}
```

```
class Array2D {  
    + rows: int  
    + cols: int  
    + data: List<Any>  
    + get_val(r: int, c: int): Any  
    + set_val(r: int, c: int, value: Any): void  
}
```

```
class UserHashTable {
```

```

- _SIZE: int
- _table: List<UserNode>
+ hash_function(key: String): int
+ insert(username: String, value: UserData): void
+ get(username: String): UserData
+ contains(username: String): bool
+ remove(username: String): bool
+ display(): void
+ get_table(): List<UserNode>
+ get_all(): List<UserData>
}

```

```

class TicketLinkedList {
- _head: TicketNode
- _tail: TicketNode
+ add_ticket(ticket: TicketData): void
+ find_ticket(ticket_id: String): TicketNode
+ remove_ticket(ticket_id: String): bool
+ find_by_user(user_id: String): List<TicketData>
+ traverse(): void
+ get_head(): TicketNode
+ get_tail(): TicketNode
+ clear(): void
}

```

```

class MovieLinkedList {
- _head: MovieNode
- _tail: MovieNode
+ add_movie(movie: MovieData): void
+ search_movie(title: String): MovieNode
+ search_id(movie_id: String): MovieNode
+ remove_movie(movie_id: String): bool
+ sort_by_revenue_logic(): void
+ traverse(): void
+ get_head(): MovieNode
}

```

```

class ShowtimeLinkedList {
- _head: ShowtimeNode
- _tail: ShowtimeNode
+ add_showtime(st: Showtime): void
+ find_showtime(showtime_id: String): ShowtimeNode
+ find_by_movie(movie_id: String): List<Showtime>
+ remove_showtime(showtime_id: String): bool
}

```

```

+ traverse(): void
+ get_head(): ShowtimeNode
}

class RoomLinkedList {
- _head: RoomNode
- _tail: RoomNode
+ add_room(room: Room): void
+ find_room(room_id: String): RoomNode
+ remove_room(room_id: String): bool
+ traverse(): void
+ get_head(): RoomNode
}

```

```

// Mối quan hệ kế thừa
UserNode --> Node
TicketNode --> Node
MovieNode --> Node
ShowtimeNode --> Node
RoomNode --> Node

```

TẦNG 4. DATA MODEL LAYER

```

class SeatStatus <<enum>> {
+ EMPTY: int = 0
+ RESERVED: int = 1
+ BOOKED: int = 2
+ BLOCKED: int = -1
}

class UserData <<entity>> {
- _username: String
- _password: String
- _role: String
- _user_id: String

+ get_username(): String
+ get_password(): String
+ get_role(): String
+ get_user_id(): String
+ check_password(password: String): bool
}

```

```
class TicketData <<entity>> {  
    - _ticket_id: String  
    - _user_id: String  
    - _movie_id: String  
    - _seat_id: String  
    - _status: String  
    - _showtime_id: String  
    - _room_id: String  
    - _price: double  
    - _booking_time: String  
  
    + get_ticket_id(): String  
    + get_user_id(): String  
    + get_movie_id(): String  
    + get_seat_id(): String  
    + get_status(): String  
    + get_showtime_id(): String  
    + get_room_id(): String  
    + get_price(): double  
    + get_booking_time(): String  
    + set_status(status: String): void  
}
```

```
class MovieData <<entity>> {  
    - _movie_id: String  
    - _title: String  
    - _genre: String  
    - _duration: int  
    - _description: String  
    - _base_price: double  
    - _poster_path: String  
    - _revenue: double  
  
    + get_movie_id(): String  
    + get_title(): String  
    + get_genre(): String  
    + get_duration(): int  
    + get_description(): String  
    + get_base_price(): double  
    + get_poster_path(): String  
    + get_revenue(): double  
  
    + set_title(title: String): void  
    + set_genre(genre: String): void
```



```

+ set_duration(duration: int): void
+ set_description(description: String): void
+ set_base_price(base_price: double): void
+ set_poster_path(poster_path: String): void
+ set_revenue(revenue: double): void
+ add_revenue(amount: double): void
}

```

```

class SeatMatrix {
  - _rows: int
  - _cols: int
  - _seats: Array2D

  + get_rows(): int
  + get_cols(): int
  + generate_seat_id(r: int, c: int): String
  + check_status(r: int, c: int): int
  + reserve_seat(r: int, c: int): bool
  + book_seat(r: int, c: int): bool
  + release_seat(r: int, c: int): void
  + get_seats(): Array2D
  + load_matrix(seats_array: List<List<int>>): void
  + set_seat_status(r: int, c: int, status: int): bool
  + reset_all(): void
}

```

```

class Room <<entity>> {
  - _room_id: String
  - _room_name: String
  - _rows: int
  - _cols: int
  - _capacity: int

  + get_room_id(): String
  + get_room_name(): String
  + get_rows(): int
  + get_cols(): int
  + get_capacity(): int
  + set_room_name(new_name: String): void
}

```

```

class Showtime <<entity>> {
  - _showtime_id: String
  - _movie_id: String

```

```

- _start_time: String
- _room_id: String
- _seat_matrix: SeatMatrix

+ get_showtime_id(): String
+ get_movie_id(): String
+ get_start_time(): String
+ get_room_id(): String
+ get_seat_matrix(): SeatMatrix
+ set_start_time(new_time: String): void
+ get_available_seats(): int
}

class FileIOHandler {
+ base_path: String
+ users_file: String
+ movies_file: String
+ rooms_file: String
+ showtimes_file: String
+ tickets_file: String

+ load_users(table: UserHashTable): void
+ save_users(table: UserHashTable): void
+ load_ui_config(): Map<String, List<String>>
+ save_ui_config(slider_titles: List<String>, list_titles: List<String>): void
+ load_movies(movie_list: MovieLinkedList): void
+ save_movies(movie_list: MovieLinkedList): void
+ load_rooms(room_list: RoomLinkedList): void
+ save_rooms(room_list: RoomLinkedList): void
+ load_showtimes(showtime_list: ShowtimeLinkedList): void
+ save_showtimes(showtime_list: ShowtimeLinkedList): void
+ load_tickets(ticket_list: TicketLinkedList): void
+ save_tickets(ticket_list: TicketLinkedList): void
+ loadData(users: UserHashTable, movies: MovieLinkedList, rooms: RoomLinkedList,
showtimes: ShowtimeLinkedList, tickets: TicketLinkedList): void
+ saveData(users: UserHashTable, movies: MovieLinkedList, rooms: RoomLinkedList,
showtimes: ShowtimeLinkedList, tickets: TicketLinkedList): void
}

```